FIAP ON

SAFRA DO CONHECIMENTO
CIBERNÉTICO - PRAGAS VIRTUAIS

IMPLEMENTANDO ALGORITMOS DE MACHINE LEARNING COM SCIKIT-LEARN

3

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE FIGURAS

Figura 1 - Logo scikit-learn 6
Figura 2 - Metodologia CRISP-DM 9
Figura 3 - Site scikit-learn 10
Figura 4 - Resultado importe Iris 12
Figura 5 - Resultado importe digits 13
Figura 6 - Resultado plot 2D e 3D 17
Figura 7 - Exibe df wine 18
Figura 8 - Resultado normalização 20
Figura 9 - Resultado valores ausentes 21
Figura 10 - Resultado imputação de valores ausentes 23
Figura 11 - Dataset produção agrícola 25
Figura 12 - Aplicação LabelEncoder 26
Figura 13 - Aplicação OneHotEncoder 27
Figura 14 - Aplicação OrdinalEncoder 28
Figura 15 - Aplicação train_test_split 30
Figura 16 - Utilização dos métodos fit predict 31
Figura 17 - Utilização dos métodos score 32
Figura 18 - Dados make_blob 38
Figura 19 - Resultado Agrupamento K-Means e DBSCAN 39
Figura 20 - Matriz de confusão 40
Figura 21 - Matriz de confusão (2) 42

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 - Exemplo import dataset Iris 11
Código-fonte 2 - Exemplo digits 12
Código-fonte 3 - Exemplo OpenML 13
Código-fonte 4 - Exemplo make_classificaiton.....15
Código-fonte 5 - Exemplo plot 2D e 3D 16
Código-fonte 6 - Carrega dataset wine.....17
Código-fonte 7 - Aplicação StandardScaler e MinMaxScaler 19
Código-fonte 8 - Adiciona valores ausentes 21
Código-fonte 9 - Imputação de valores ausentes 22
Código-fonte 10 - Criação dataset.....24
Código-fonte 11 - Aplicação LabelEncoder 25
Código-fonte 12 - Aplicação OneHotEncoder.....26
Código-fonte 13 - Aplicação OrdinalEncoder 27
Código-fonte 14 - Aplicação train_test_split 29
Código-fonte 15 - Dataset Wine para classificação.....33
Código-fonte 16 - Treinamento dataset Wine para classificação 34
Código-fonte 17 - Testando diferentes algoritmos de classificação 35
Código-fonte 18 - Dataset California para regressão 36
Código-fonte 19 - Testando diferentes algoritmos de regressão.....36
Código-fonte 20 - Dataset make_blob.....37
Código-fonte 21 - Testando diferentes algoritmos de agrupamento.....38
Código-fonte 22 - Testando diferentes algoritmos de agrupamento (2) 40
Código-fonte 23 - Treina com knn o wines.....41
Código-fonte 24 - Métricas de performance 42
Código-fonte 25 - Treina com Random Forest Regressor o California.....43
Código-fonte 26 - Treina com Random Forest Regressor o California (2) 44

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LISTA DE COMANDOS DE PROMPT DO SISTEMA OPERACIONAL

Comando de prompt 1 - Comando para instalação da biblioteca.....7
Comando de prompt 2 - Comando para instalação da biblioteca usando venv 8
Comando de prompt 3 - Comando para instalação da biblioteca usando conda 8
Comando de prompt 4 - Comando para verificar instalação 8

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SUMÁRIO

1 IMPLEMENTANDO ALGORITMOS DE MACHINE LEARNING COM SCIKIT-LEARN 6
1.1 Introdução 6
1.2 Instalação 7
1.3 Scikit-learn em projetos de Machine Learning.....8
1.4 Módulos scikit-learn.....10
1.5 Datasets 10
1.6 Pré-processamento 17
1.7 Módulo model_selection.....28
1.8 Estimadores e treinamento de modelos 30
1.9 Ferramentas para avaliação e otimização.....39
CONCLUSÃO.....45
REFERÊNCIAS.....46
GLOSSÁRIO 47

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1 IMPLEMENTANDO ALGORITMOS DE MACHINE LEARNING COM SCIKIT-LEARN

1.1 Introdução

O scikit-learn (também conhecido como sklearn) é uma biblioteca de código aberto (*open source*) para a linguagem Python. Essa biblioteca fornece uma ampla gama de ferramentas e algoritmos de Aprendizado de Máquina de forma acessível e eficiente.

Figura 1 - Logo scikit-learn
Fonte: scikit-learn (2024)

Podemos dizer que o scikit-learn é a ferramenta mais popular e utilizada em Aprendizado de Máquina, e existem alguns motivos para isso, tais como:

- **Simplicidade e consistência:** todos os algoritmos no scikit-learn compartilham uma interface consistente, que segue um padrão de métodos como fit(), predict() e transform(). Isso facilita a troca e experimentação de diferentes algoritmos, promovendo um ambiente de desenvolvimento rápido e eficiente.
- **Compatibilidade e integração:** o scikit-learn é projetado para interoperabilidade com outras bibliotecas do ecossistema Python, como Pandas para manipulação de dados, Numpy para operações numéricas e Matplotlib para visualização. Essa integração permite uma fácil manipulação

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

e transformação de dados, assim como a visualização de resultados de modelos.

- **Foco em performance e eficiência computacional:** embora Python seja uma linguagem interpretada, a scikit-learn faz uso extensivo de operações vetorizadas através de Numpy e implementações otimizadas em Cython — uma linguagem de programação que facilita a escrita de extensões em C para Python — para acelerar o processamento de dados e a execução de algoritmos. Isso proporciona uma execução eficiente de tarefas complexas de Machine Learning, como treinamento de modelos e ajuste de hiperparâmetros.

Esta biblioteca é amplamente utilizada tanto na academia quanto na indústria, sendo uma escolha padrão para cientistas de dados e engenheiros de Machine Learning que buscam implementar modelos preditivos robustos e otimizados. O scikit-learn oferece funcionalidades para tarefas de classificação, regressão, clustering, redução de dimensionalidade, pré-processamento de dados e avaliação de modelos. Nas próximas seções vamos conhecer um pouco dessas funcionalidades, mas antes é necessário preparar e configurar adequadamente o ambiente de desenvolvimento.

1.2 Instalação

O scikit-learn é uma biblioteca de Python, portanto, a primeira etapa é garantir que você tenha Python instalado em seu sistema. Com o Python instalado, podemos utilizar o gerenciador de pacotes *pip* para realizar uma instalação da biblioteca. Para isso, abra o terminal ou prompt de comando e execute o seguinte comando:

```
pip install scikit-learn
```

Comando de prompt 1 - Comando para instalação da biblioteca
Fonte: Elaborado pelo autor (2024)

Este comando irá baixar e instalar o scikit-learn juntamente com suas dependências. É importante verificar se todas as dependências, como NumPy e SciPy, estão atualizadas para evitar conflitos. Por esse motivo, é importante criar um

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

ambiente virtual para seus projetos de Machine Learning para gerenciar dependências de forma isolada e evitar conflitos entre bibliotecas. Você pode criar um ambiente virtual utilizando venv ou conda (para usuários Anaconda):

- **Venv:**

```
python -m venv sklearn-env
sklearn-env\Scripts\activate  # activate
pip install -U scikit-learn
```

Comando de prompt 2 - Comando para instalação da biblioteca usando venv
Fonte: Elaborado pelo autor (2024)

- **Conda:**

```
conda create -n sklearn-env -c conda-forge scikit-learn
conda activate sklearn-env
```

Comando de prompt 3 - Comando para instalação da biblioteca usando conda
Fonte: Elaborado pelo autor (2024)

Podemos verificar se a instalação está correta verificando a versão, para isso, abra o terminal ou prompt de comando e execute o seguinte comando:

```
python -c "import sklearn; print(sklearn.__version__)"
```

Comando de prompt 4 - Comando para verificar instalação
Fonte: Elaborado pelo autor (2024)

Desse modo, temos o retorno da versão instalada em seu computador. Com a versão instalada, podemos prosseguir e explorar os recursos que a biblioteca oferece em projetos de Ciência de Dados e Machine Learning.

1.3 Scikit-learn em projetos de Machine Learning

Projetos de Ciência de Dados e Aprendizado de Máquina não se resumem apenas ao treinamento de um algoritmo para obtenção de um modelo de Machine Learning. Eles envolvem uma série de etapas para garantir a eficácia e a escalabilidade da solução desenvolvida. Desde a coleta e preparação de dados, passando pela escolha e ajuste de modelos, até a validação e implementação desses

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

modelos em um ambiente de produção, cada etapa desempenha um papel essencial na construção de um pipeline robusto e eficiente.

Uma das abordagens metodológicas utilizadas para organizar essas etapas é o CRISP-DM (Cross Industry Standard Process for Data Mining). Esta metodologia orienta o desenvolvimento de projetos de Ciência de Dados e Aprendizado de Máquina em seis fases principais: Entendimento do Negócio, Entendimento dos Dados, Preparação dos Dados, Modelagem, Avaliação e Deploy. Seguindo essa estrutura, os profissionais garantem que todas as fases do projeto sejam consideradas, minimizando riscos e maximizando a eficácia dos resultados obtidos. O scikit-learn é projetado para fornecer suporte em todas essas etapas, facilitando a construção de soluções eficazes e escaláveis.

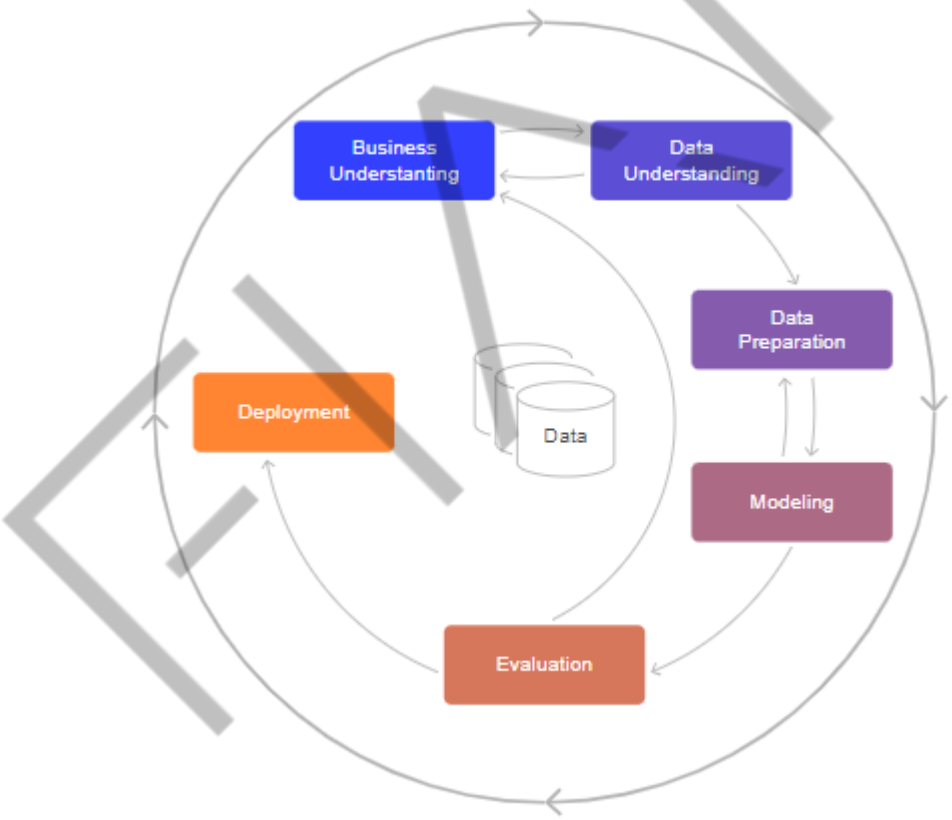

Figura 2 - Metodologia CRISP-DM
Fonte: Medium (2019), adaptado por FIAP (2024)

Dica: para conhecer mais sobre a metodologia CRISP-DM acesse: <https://www.preditiva.ai/blog/entenda-o-crisp-dm-suas-etapas-e-como-de-fato-gerar-valor-com-essa-metodologia>.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

1.4 Módulos scikit-learn

A documentação oficial do scikit-learn, disponível no site <https://scikit-learn.org>, é uma excelente fonte de informações detalhadas sobre cada um dos recursos desta biblioteca, incluindo tutoriais, exemplos de código e explicações.

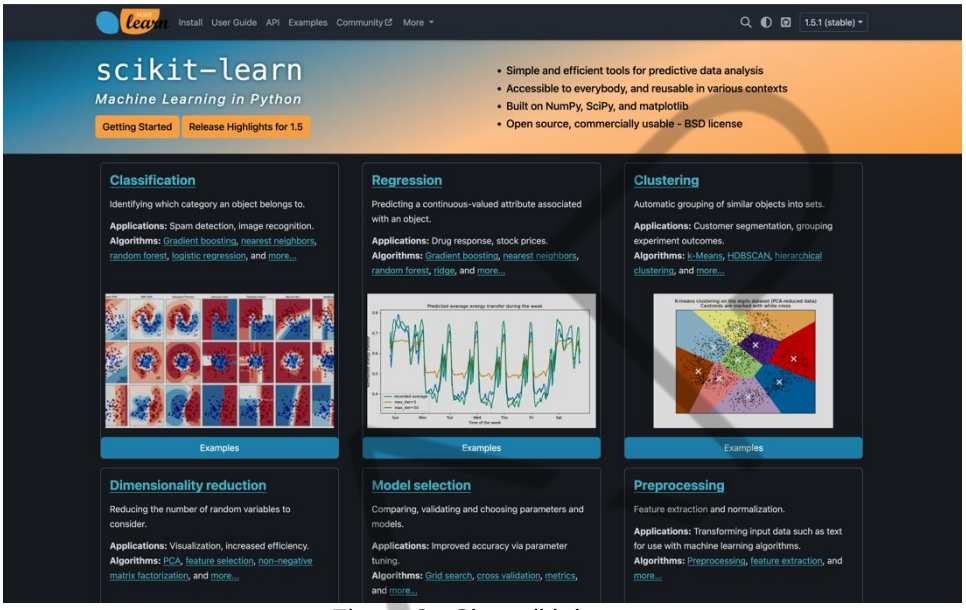

Figura 3 - Site scikit-learn
Fonte: Elaborado pelo autor (2024)

Dica: para se aprofundar mais, acesse o site da documentação oficial: <https://scikit-learn.org/stable/>.

A seguir, vamos explorar os principais módulos do scikit-learn e suas funcionalidades.

1.5 Datasets

O módulo datasets fornece diversas funções para carregar conjuntos de dados clássicos no aprendizado de Machine Learning, além de ser possível gerar conjuntos de dados sintéticos para teste e validação de algoritmos de Aprendizado de Máquina. Esse módulo pode ser dividido em três partes, sendo cada uma delas adequada para diferentes necessidades de projeto:

- **Toy Datasets:** são conjuntos de dados incorporados diretamente na biblioteca e podem ser facilmente carregados na memória para uso, além

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

disso, eles são pequenos e fáceis de entender, ideais para aprendizado e demonstração. Alguns exemplos são:

- **Iris Dataset (load_iris):** conjunto de dados de classificação para análise de flores.
- **Diabetes Dataset (load_diabetes):** conjunto de dados de regressão para prever a progressão da diabetes.

O carregamento do conjunto de dados é simples, após importar do conjunto de dados, ele retorna um objeto Bunch, que é similar a um dicionário Python. Esse objeto contém chaves como data, target, feature_names e DESCR, facilitando o acesso às informações do dataset, como demonstra o código a seguir:

```
from sklearn.datasets import load_iris

# Carrega o dataset
iris = load_iris()

# Acessar dados e rótulos
X, y, name = iris.data, iris.target, iris.feature_names

print("Características:\n", X[:5])
print("Rótulos:\n", y[:5])
print("Nome das características:\n", name)

# Acessar metadados
print("Metadados:\n", iris.DESCR)
```

Código-fonte 1 - Exemplo import dataset Iris
Fonte: Elaborado pelo autor (2024)

O resultado esperado é que seja importado o dataset Iris, e também que ele exiba os dados, rótulos e características do conjunto de dados, como mostra a figura “Resultado importe Iris”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Características:
 [[5.1 3.5 1.4 0.2]
 [4.9 3.  1.4 0.2]
 [4.7 3.2 1.3 0.2]
 [4.6 3.1 1.5 0.2]
 [5.  3.6 1.4 0.2]]
Rótulos:
 [0 0 0 0 0]
Nome das características:
 ['sepal length (cm)', 'sepal width (cm)', 'petal length (cm)', 'petal width (cm)']
Metadados:
 .. _iris_dataset:

Iris plants dataset
--------------------

**Data Set Characteristics:**

    :Number of Instances: 150 (50 in each of three classes)
    :Number of Attributes: 4 numeric, predictive attributes and the class
    :Attribute Information:
        - sepal length in cm
        - sepal width in cm
        - petal length in cm
        - petal width in cm
        - class:
                - Iris-Setosa
                - Iris-Versicolour
                - Iris-Virginica

    :Summary Statistics:
```

Figura 4 - Resultado importe Iris
Fonte: Elaborado pelo autor (2024)

De modo simples e semelhante, podemos utilizar o Matplotlib para visualizar de forma gráfica os dados e realizar nossas análises estatísticas. O código a seguir realiza o carregamento e exibição do dataset digits de imagens de 8x8 pixels de tamanho.

```
from sklearn.datasets import load_digits
import matplotlib.pyplot as plt

# Carregar o conjunto de dados
digits = load_digits()

# Exibir algumas imagens de dígitos
fig, axes = plt.subplots(1, 4, figsize=(10, 3))
for ax, image, label in zip(axes, digits.images, digits.target):
    ax.set_axis_off()
    ax.imshow(image, cmap=plt.cm.gray_r, interpolation='nearest')
    ax.set_title(f'Target: {label}')
plt.show()
```

Código-fonte 2 - Exemplo digits
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

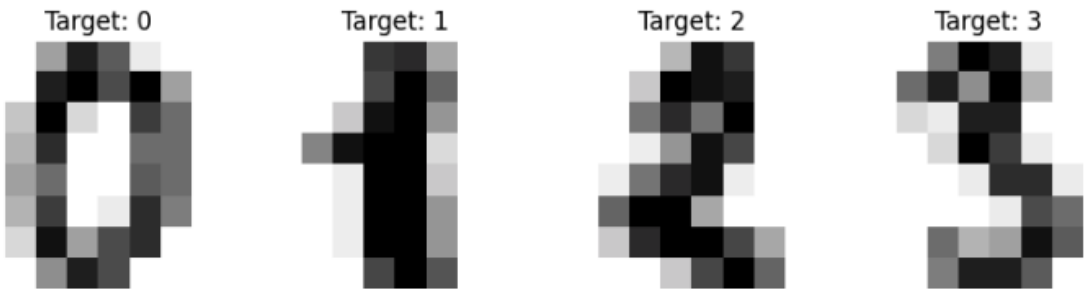

Figura 5 - Resultado importe digits
Fonte: Elaborado pelo autor (2024)

- **Conjuntos de dados externos:** é possível obter dados a partir de várias fontes, como URLs diretas, formatos específicos como SVMLight/LibSVM, ou plataformas colaborativas como o OpenML. A função fetch_openml é particularmente útil para carregar conjuntos de dados diretamente do OpenML, uma plataforma colaborativa de dados de Machine Learning que contém uma vasta coleção de conjuntos de dados utilizados pela comunidade global de cientistas de dados para *benchmarking* e experimentação.

```
from sklearn.datasets import fetch_openml

# Carrega o dataset fornecido pelo site do openml
data = fetch_openml('Worldwide-Meat-Consumption',
as_frame=True)

# Exibindo a descrição do dataset
print(data.DESCR)

# Exibindo os nomes das características
print(data.feature_names)

# Exibindo as primeiras linhas do DataFrame
data.frame.head()
```

Código-fonte 3 - Exemplo OpenML
Fonte: Elaborado pelo autor (2024)

O trecho de código demonstra como usar a função fetch_openml para carregar o conjunto de dados “Worldwide Meat Consumption” da OECD, que está disponível no OpenML. Este conjunto de dados contém informações sobre o consumo de carne em vários países ao redor do mundo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Dica:

- Documentação oficial do OpenML: <https://openml.org/>.

- Relação ao OECD do dataset “Worldwide Meat Consumption”: <https://www.oecd.org/en/data/indicators/meat-consumption.html>.

- **Geradores de conjuntos de dados sintéticos:** é possível criar dados artificiais que seguem distribuições específicas, ou seja, permitem controlar características estatísticas dos dados, como correlação e relevância das features. Esses dados são úteis para realizar testes controlados, como, por exemplo, o impacto da escala do dataset (número de amostras e features) em modelos de Machine Learning. Alguns exemplos são:
  - **make_classification:** gera conjuntos de dados para problemas de classificação, onde é possível especificar o número de amostras, características informativas, características redundantes e clusters.
  - **make_regression:** gera conjuntos de dados para problemas de regressão, com controle sobre o número de amostras, features e nível de ruído.
  - **make_blobs:** gera clusters de dados gaussianos para tarefas de clusterização.
  - **make_moons e make_circles:** gera dados para problemas de classificação não lineares.

A seguir, iremos utilizar make_classification para gerar um conjunto de dados de 300 amostras com três features e três classes para classificação. Altere o parâmetro random_state para criar conjuntos de dados distintos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.datasets import make_classification
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

X, y = make_classification(n_samples=300,        # Número
de amostras
                           n_features=3,         # Número
total de características
                           n_informative=3,      # Número
de características informativas
                           n_redundant=0,        # Número
de características redundantes
                           n_clusters_per_class=1,      #
Número de clusters por classe
                           n_classes=3,          # Número
de classes
                           random_state=0)      # Semente
para reprodução dos resultados

# Criando um DataFrame para visualização
df = pd.DataFrame(X, columns=['Feature 1', 'Feature 2',
'Feature 3'])
df['Target'] = y

df.head()  # Mostrando as primeiras linhas do DataFrame
```

Código-fonte 4 - Exemplo make_classificaiton
Fonte: Elaborado pelo autor (2024)

Agora, vamos utilizar o Matplotlib para visualizar o conjunto de dados criado, podemos visualizar em 2D ou em 3D.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
    # Visualização 2D das duas primeiras características
    plt.figure(figsize=(8, 6))
    plt.scatter(df['Feature 1'], df['Feature 2'],
c=df['Target'], cmap='viridis', edgecolor='k', s=50)
    plt.xlabel('Feature 1')
    plt.ylabel('Feature 2')
    plt.title('2D Scatter Plot of Classification Data with 3
Classes')
    plt.colorbar(label='Target Class')
    plt.grid(True)
    plt.show()

    # Criando o gráfico 3D
    fig = plt.figure(figsize=(10, 7))
    ax = fig.add_subplot(111, projection='3d')

    # Usando uma paleta de cores
    colors = plt.cm.viridis(df['Target'] /
np.max(df['Target']))

    # Adicionando o scatter plot 3D
    scatter = ax.scatter(df['Feature 1'], df['Feature 2'],
df['Feature 3'], c=colors, s=100, edgecolor='k', alpha=0.7)

    # Adicionando rótulos aos eixos
    ax.set_xlabel('Feature 1')
    ax.set_ylabel('Feature 2')
    ax.set_zlabel('Feature 3')

    # Definindo o título
    ax.set_title('3D Plot')

    # Exibindo o gráfico
    plt.show()
```

Código-fonte 5 - Exemplo plot 2D e 3D
Fonte: Elaborado pelo autor (2024)

É esperado como resultado a exibição do conjunto de dados criado, conforme ilustrado na figura “Resultado plot 2D e 3D”.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

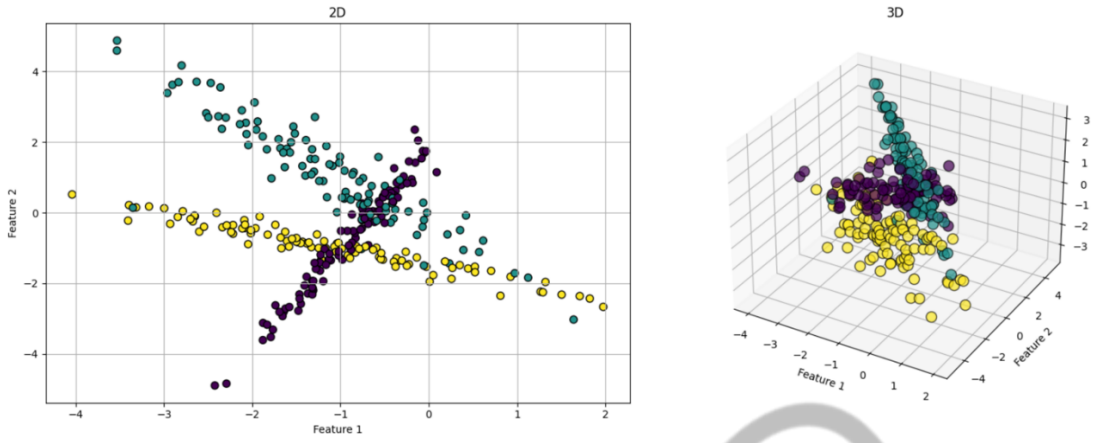

Figura 6 - Resultado plot 2D e 3D
Fonte: Elaborado pelo autor (2024)

1.6 Pré-processamento

O módulo *preprocessing* é utilizado para manipular e transformar dados antes de aplicá-los a algoritmos de aprendizado de máquina. Entre as diversas possibilidades de pré-processamento que a biblioteca disponibiliza, estão as técnicas de escalonamento de características, imputação de valores ausentes, codificação de variáveis categóricas e normalização de dados. Iremos utilizar um conjunto de dados real para demonstrar como aplicar as principais técnicas de pré-processamento disponíveis no scikit-learn. Isso ajudará a entender na prática como essas transformações afetam os dados e por que são importantes.

O dataset Wine do sklearn é um conjunto de dados que contém informações de diferentes vinhos, tais como teor alcoólico, acidez, magnésio, fenóis, entre outras.

```
from sklearn.datasets import load_wine
import pandas as pd
import numpy as np

# Carregando o dataset
wine = load_wine()
X = wine.data  # Características
y = wine.target  # Rótulos

# Convertendo para DataFrame para melhor visualização
df = pd.DataFrame(X, columns=wine.feature_names)
df['target'] = y
df.head()
```

Código-fonte 6 - Carrega dataset wine
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Na figura “Exibe df wine”, conseguimos visualizar apenas alguns dos atributos, note que existem escalas de valores diferentes.

magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline
127.0	2.80	3.06	0.28	2.29	5.64	1.04	3.92	1065.0
100.0	2.65	2.76	0.26	1.28	4.38	1.05	3.40	1050.0
101.0	2.80	3.24	0.30	2.81	5.68	1.03	3.17	1185.0
113.0	3.85	3.49	0.24	2.18	7.80	0.86	3.45	1480.0
118.0	2.80	2.69	0.39	1.82	4.32	1.04	2.93	735.0

Figura 7 - Exibe df wine
Fonte: Elaborado pelo autor (2024)

- **Escalonamento e normalização:** em muitos casos, as variáveis dos conjuntos de dados possuem diferentes escalas, o que pode impactar negativamente a eficácia do modelo de ML. O dataset Wine tem essa característica com variáveis em escalas diferentes (algumas variam entre 0 e 10, outras entre 0 e 1000). Isso pode causar problemas para algoritmos que são sensíveis à escala, como *SVM* e *KNN*. Vamos aplicar dois métodos de escalonamento:
  - **StandardScaler:** transforma os dados para que possuem média 0 e desvio padrão 1. Utilizado quando os dados seguem uma distribuição aproximadamente normal.
  - **MinMaxScaler:** transforma os dados para que fiquem dentro de um intervalo específico, geralmente [0, 1]. Utilizado em algoritmos baseados em distância.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.preprocessing import StandardScaler, MinMaxScaler
import pandas as pd

# Aplicando StandardScaler
scaler_standard = StandardScaler()
X_scaled = scaler_standard.fit_transform(X)

# Aplicando MinMaxScaler
scaler_minmax = MinMaxScaler()
X_minmax_scaled = scaler_minmax.fit_transform(X)

# Mostrando as primeiras linhas dos dados escalados
df_scaled = pd.DataFrame(X_scaled, columns=wine.feature_names)
df_scaled.head()

df_minmax = pd.DataFrame(X_minmax_scaled, columns=wine.feature_names)
df_minmax.head()
```

Código-fonte 7 - Aplicação StandardScaler e MinMaxScaler
Fonte: Elaborado pelo autor (2024)

A figura “Resultado normalização” exibe os resultados dos dois métodos de escalonamento. No primeiro bloco de dados, após a aplicação do StandardScaler, as variáveis escaladas possuem média 0 e desvio padrão 1, o que é importante para modelos que assumem uma distribuição gaussiana ou precisam de dados centralizados. No segundo bloco de dados, as colunas escaladas pelo MinMaxScaler foram normalizadas para o intervalo [0, 1]. Aplicado em algoritmos que utilizam distâncias ou gradientes, garantindo que as features estejam na mesma escala, sem permitir que uma tenha influência indevida no modelo por conta de sua magnitude original.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline
1.913905	0.808997	1.034819	-0.659563	1.224884	0.251717	0.362177	1.847920	1.013009
0.018145	0.568648	0.733629	-0.820719	-0.544721	-0.293321	0.406051	1.113449	0.965242
0.088358	0.808997	1.215533	-0.498407	2.135968	0.269020	0.318304	0.788587	1.395148
0.930918	2.491446	1.466525	-0.981875	1.032155	1.186068	-0.427544	1.184071	2.334574
1.281985	0.808997	0.663351	0.226796	0.401404	-0.319276	0.362177	0.449601	-0.037874

magnesium	total_phenols	flavanoids	nonflavanoid_phenols	proanthocyanins	color_intensity	hue	od280/od315_of_diluted_wines	proline
0.619565	0.627586	0.573840	0.283019	0.593060	0.372014	0.455285	0.970696	0.561341
0.326087	0.575862	0.510549	0.245283	0.274448	0.264505	0.463415	0.780220	0.550642
0.336957	0.627586	0.611814	0.320755	0.757098	0.375427	0.447154	0.695971	0.646933
0.467391	0.989655	0.664557	0.207547	0.558360	0.556314	0.308943	0.798535	0.857347
0.521739	0.627586	0.495781	0.490566	0.444795	0.259386	0.455285	0.608059	0.325963

Figura 8 - Resultado normalização
Fonte: Elaborado pelo autor (2024)

- **Imputação de valores ausentes:** quando lidamos com dados reais, muitas vezes encontramos valores ausentes. Esses valores precisam ser tratados para que o modelo de Machine Learning funcione corretamente. O *sklearn.preprocessing* oferece duas abordagens principais para imputar (preencher) valores ausentes:
  - **SimpleImputer:** substitui os valores ausentes por uma estratégia estatística simples, como a média, mediana, moda, ou um valor constante fornecido. É um algoritmo rápido e barato computacionalmente, porém, ao utilizá-lo para substituir valores ausentes por uma média ou mediana, esse algoritmo pode ignorar correlações ou padrões complexos nos dados.
  - **KNNImputer:** utiliza o algoritmo de K-Nearest Neighbors (KNN) para estimar valores ausentes com base em valores vizinhos no conjunto de dados. Essa técnica faz a imputação com base em padrões de correlação entre as variáveis, por isso é computacionalmente mais cara e pode ser sensível ao valor de k. Além disso, pode ser ineficaz se não houver correlação entre as variáveis ou se houver muitas dimensões com valores ausentes.

Em nosso exemplo, o dataset wine não contém valores ausentes, por esse motivo, é utilizado o Numpy para introduzir alguns valores ausentes artificialmente no dataset e demostrar as técnicas de imputação de valores ausentes.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import numpy as np
import pandas as pd

# Adiciona valores ausentes para demonstrar de exemplo
X_missing = X.copy()
X_missing.loc[0:4, 'alcohol'] = np.nan  # Primeiros 5
valores ausentes
X_missing.loc[0:2, 'malic_acid'] = np.nan  # Primeiros 3
valores ausentes

# Visualizar os dados com valores ausentes
# X_missing.isnull().sum()
X_missing.head()
```

Código-fonte 8 - Adiciona valores ausentes
Fonte: Elaborado pelo autor (2024)

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium
0	NaN	NaN	2.43	15.6	127.0
1	NaN	NaN	2.14	11.2	100.0
2	NaN	NaN	2.67	18.6	101.0
3	NaN	1.95	2.50	16.8	113.0
4	NaN	2.59	2.87	21.0	118.0

Figura 9 - Resultado valores ausentes
Fonte: Elaborado pelo autor (2024)

Com o conjunto de dados modificado e armazenado na variável X_missing, vamos aplicar as técnicas de imputação de dados e avaliar seus resultados.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.impute import SimpleImputer, KNNImputer

# Aplicando SimpleImputer (substitui os valores ausentes pela média)
simple_imputer = SimpleImputer(strategy='mean')
X_simple_imputed = simple_imputer.fit_transform(X_missing)

# Aplicando KNNImputer (substitui os valores ausentes usando os 5 vizinhos mais próximos)
knn_imputer = KNNImputer(n_neighbors=5)
X_knn_imputed = knn_imputer.fit_transform(X_missing)

# Converter de volta para DataFrame para facilitar a visualização
X_simple_imputed = pd.DataFrame(X_simple_imputed, columns=wine.feature_names)

X_knn_imputed = pd.DataFrame(X_knn_imputed, columns=wine.feature_names)

# Exibir resultado
print("SimpleImputer:")
display(X_simple_imputed.head())

print("\nKNNImputer:")
display(X_knn_imputed.head())
```

Código-fonte 9 - Imputação de valores ausentes
Fonte: Elaborado pelo autor (2024)

A figura “Resultado imputação de valores ausentes” exibe os resultados dos métodos de imputação de dados ausentes. No SimpleImputer foi utilizado o valor da média, e no caso do KNNImputer foi utilizado a substituição pelos cinco vizinhos mais próximos.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

SimpleImputer:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium
0	12.982139	2.342971	2.43	15.6	127.0
1	12.982139	2.342971	2.14	11.2	100.0
2	12.982139	2.342971	2.67	18.6	101.0
3	12.982139	1.950000	2.50	16.8	113.0
4	12.982139	2.590000	2.87	21.0	118.0

KNNImputer:

	alcohol	malic_acid	ash	alcalinity_of_ash	magnesium
0	13.988	2.624	2.43	15.6	127.0
1	14.176	2.092	2.14	11.2	100.0
2	13.768	1.710	2.67	18.6	101.0
3	14.006	1.950	2.50	16.8	113.0
4	13.272	2.590	2.87	21.0	118.0

Figura 10 - Resultado imputação de valores ausentes
Fonte: Elaborado pelo autor (2024)

- **Codificação de variáveis categóricas:** em projetos de Ciência de Dados e Aprendizado de Máquina, nos deparamos com variáveis categóricas, ou seja, que assumem valores discretos ou qualitativos, como “gênero”, “cor”, “categoria de produto”, entre outros. De forma geral, os algoritmos de Machine Learning esperam que as entradas sejam numéricas. Logo, essas variáveis precisam ser transformadas antes que possam ser utilizadas como input para os modelos. Esse processo é conhecido como codificação de variáveis categóricas. No entanto, a maneira como essas variáveis são codificadas pode afetar o desempenho do modelo, especialmente em termos de eficiência e capacidade de generalização. Por essa razão, existem diferentes formas de transformar variáveis categóricas em valores numéricos. Os métodos mais comuns e utilizados com o scikit-learn são:
    - **LabelEncoder:** converte cada categoria em um valor inteiro único. Cada categoria recebe um número inteiro diferente, e os valores numéricos não possuem relação direta entre si além de representar categorias distintas. As categorias são ordenadas alfabeticamente e cada categoria recebe um valor correspondente.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **OneHotEncoder:** cria uma nova coluna binária (0 ou 1) para cada categoria, sem considerar qualquer hierarquia entre elas. Essa técnica é especialmente útil quando não há relação ordinal entre as categorias.
- **OrdinalEncoding:** atribui valores inteiros às categorias, mas preserva uma relação de ordem intrínseca entre elas. Este método é apropriado quando as categorias possuem uma ordem natural (por exemplo, “pequeno”, “médio”, “grande”). A ordem das categorias deve ser explicitamente definida durante o processo de codificação, garantindo que os valores numéricos respeitem a hierarquia natural dos dados.

Neste momento, iremos praticar as técnicas de codificação em um conjunto de dados sobre Agronegócio com informações sobre diferentes tipos de culturas, as regiões onde são cultivadas, e algumas características quantitativas. Criamos esse conjunto de dados utilizando o Pandas.

```
import pandas as pd

# Criando o conjunto de dados fictício
data = {
    'Cultura': ['Milho', 'Soja', 'Café', 'Cana-de-
açúcar', 'Arroz', 'Algodão'],
    'Região': ['Norte', 'Sul', 'Sudeste', 'Nordeste',
'Centro-Oeste', 'Nordeste'],
    'Produção (toneladas)': [5000, 12000, 8000, 15000,
11000, 7000],
    'Preço por Tonelada (R$)': [750, 1300, 2000, 600, 900,
1400],
    'Tipo de Solo': ['Argiloso', 'Arenoso', 'Misto',
'Argiloso', 'Arenoso', 'Misto']
}

# Convertendo em DataFrame
df = pd.DataFrame(data)
df.head()
```

Código-fonte 10 - Criação dataset
Fonte: Elaborado pelo autor (2024)

A figura “Dataset produção agrícola” exibe o nosso conjunto de dados fictício de produção agrícola.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Cultura	Região	Produção (toneladas)	Preço por Tonelada (R$)	Tipo de Solo
0	Milho	Norte	5000	750	Argiloso
1	Soja	Sul	12000	1300	Arenoso
2	Café	Sudeste	8000	2000	Misto
3	Cana-de-açúcar	Nordeste	15000	600	Argiloso
4	Arroz	Centro-Oeste	11000	900	Arenoso
5	Algodão	Nordeste	7000	1400	Misto

Figura 11 - Dataset produção agrícola
Fonte: Elaborado pelo autor (2024)

Aplicamos as codificações nas colunas:

- **Label Encoding:** a coluna “Cultura” será transformada em valores numéricos inteiros para facilitar o uso em modelos que esperam variáveis numéricas.
- **One-Hot Encoding:** a coluna “Região” será transformada em múltiplas colunas binárias, uma para cada região, refletindo a diversidade geográfica de produção.
- **Ordinal Encoding:** a coluna “Tipo de Solo” será codificada com base em uma ordem de produtividade do solo no contexto agrícola.

Começamos com a codificação Label Encoding com o trecho de código a seguir:

```
from sklearn.preprocessing import LabelEncoder

# Inicializando o LabelEncoder
label_encoder = LabelEncoder()

# Aplicando Label Encoding na coluna 'Cultura'
df['Cultura_LabelEncoded']                                    =
label_encoder.fit_transform(df['Cultura'])

df[['Cultura', 'Cultura_LabelEncoded']]
```

Código-fonte 11 - Aplicação LabelEncoder
Fonte: Elaborado pelo autor (2024)

A figura “Aplicação LabelEncoder” mostra o Label Encoding convertendo cada cultura em um número único, que será útil para modelos como árvores de decisão e

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

florestas aleatórias, onde não há problema em utilizar essa representação numérica. Note que as categorias foram codificadas de acordo com a ordem alfabética.

	Cultura	Cultura_LabelEncoded
0	Milho	4
1	Soja	5
2	Café	2
3	Cana-de-açúcar	3
4	Arroz	1
5	Algodão	0

Figura 12 - Aplicação LabelEncoder
Fonte: Elaborado pelo autor (2024)

Seguimos com a codificação One-Hot Encoding no trecho de código a seguir:

```
from sklearn.preprocessing import OneHotEncoder

# Inicializando o OneHotEncoder
onehot_encoder = OneHotEncoder(sparse_output=False)

# Aplicando One-Hot Encoding na coluna 'Região'
onehot_encoded =
onehot_encoder.fit_transform(df[['Região']])

# Convertendo para DataFrame para fácil visualização
encoded_columns =
onehot_encoder.get_feature_names_out(['Região'])
onehot_df = pd.DataFrame(onehot_encoded,
columns=encoded_columns)

# Juntando os resultados ao DataFrame original
df = pd.concat([df, onehot_df], axis=1)
df[['Região', 'Região_Centro-Oeste', 'Região_Nordeste',
'Região_Norte', 'Região_Sul', 'Região_Sudeste']]
```

Código-fonte 12 - Aplicação OneHotEncoder
Fonte: Elaborado pelo autor (2024)

A figura “Aplicação OneHotEncoder” exibe a coluna “Região” codificada usando One-Hot Encoding.

Observe que foram geradas novas colunas para representar cada uma das regiões descritas no dataset.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Região	Região_Centro-Oeste	Região_Nordeste	Região_Norte	Região_Sul	Região_Sudeste
0	Norte	0.0	0.0	1.0	0.0	0.0
1	Sul	0.0	0.0	0.0	1.0	0.0
2	Sudeste	0.0	0.0	0.0	0.0	1.0
3	Nordeste	0.0	1.0	0.0	0.0	0.0
4	Centro-Oeste	1.0	0.0	0.0	0.0	0.0
5	Nordeste	0.0	1.0	0.0	0.0	0.0

Figura 13 - Aplicação OneHotEncoder
Fonte: Elaborado pelo autor (2024)

Dica: é recomendado que você aprenda primeiro a codificação get_dumies do Pandas, pois é mais simples e rápido de usar para tarefas de codificação simples, principalmente em estágios iniciais de análise de dados, visto que converte diretamente colunas categóricas de um dataFrame em variáveis binárias (dummies).

Por fim, aplicamos a transformação *Ordinal Encoding* no trecho de código a seguir:

```
from sklearn.preprocessing import OrdinalEncoder

# Aplicando Ordinal Encoding na coluna 'Tipo de Solo'
ordinal_encoder = OrdinalEncoder(categories=[['Arenoso',
'Argiloso', 'Misto']])
df['TipoSolo_OrdinalEncoded']                          =
ordinal_encoder.fit_transform(df[['Tipo de Solo']])

df[['Tipo de Solo', 'TipoSolo_OrdinalEncoded']]
```

Código-fonte 13 - Aplicação OrdinalEncoder
Fonte: Elaborado pelo autor (2024)

A figura “Aplicação OrdinalEncoder” exibe a coluna “Tipo de Solo” codificada. Para este caso, será considerado hipoteticamente que o solo “Arenoso” é o menos produtivo, e o solo “Misto” o mais produtivo, pois o solo misto geralmente retém melhor nutrientes e água.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

	Tipo de Solo	TipoSolo_OrdinalEncoded
0	Argiloso	1.0
1	Arenoso	0.0
2	Misto	2.0
3	Argiloso	1.0
4	Arenoso	0.0
5	Misto	2.0

Figura 14 - Aplicação OrdinalEncoder
Fonte: Elaborado pelo autor (2024)

Até o momento, aprendemos as técnicas de pré-processamento mais aplicadas em projetos de Ciência de Dados e Machine Learning, aplicando as ferramentas do sklearn. Vamos conhecer outros recursos desta biblioteca.

1.7 Módulo model_selection

O módulo model_selection oferece uma série de ferramentas e utilitários para a construção de pipelines robustos de Aprendizado de Máquina. Entre as funções mais importantes, destacamos a divisão de dados, validação cruzada e ajuste de hiperparâmetros:

- **Divisão de conjuntos de dados:** em qualquer projeto de Aprendizado de Máquina, uma prática essencial é separar os dados em conjuntos de treinamento e teste. Isso permite treinar o modelo em um subconjunto dos dados e avaliá-lo em outro subconjunto, garantindo que o modelo não seja avaliado nos mesmos dados em que foi treinado.

A função train_test_split facilita esse processo, dividindo os dados em duas ou mais partes de forma aleatória ou estratificada. O parâmetro test_size define a proporção dos dados reservada para o teste, e o parâmetro random_state garante a reprodutibilidade do processo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Carregando novamente o dataset
wine = load_wine()
X = wine.data  # Características
y = wine.target  # Rótulos

# Convertendo para DataFrame para melhor visualização
df = pd.DataFrame(X, columns=wine.feature_names)
df['target'] = y

# Escolhendo X e y para fazer o split
X = df.drop('target', axis=1)
y = df['target']

# Dividindo o DataFrame em 70% para treino e 30% para
teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Conjuntos de dados
print(f'Tamanho do conjunto de dados original:
{X.shape}')
print(f'Tamanho do conjunto de treinamento:
{X_train.shape}')
print(f'Tamanho do conjunto de teste: {X_test.shape}')

# Exibir as primeiras linhas do conjunto antes e depois
do split
print("Conjunto de dados antes do split:")
display(df['target'].head())
print("\nConjunto de treinamento:")
display(y_train.head())
```

Código-fonte 14 - Aplicação train_test_split
Fonte: Elaborado pelo autor (2024)

Realizamos a divisão do dataset na proporção de 70% treino e 30% teste, para garantir reprodutibilidade da divisão foi utilizado o valor 42 como semente. Isso assegura que, quando o mesmo valor de random_state é utilizado, a divisão será a mesma a cada execução. Note que o conjunto de treinamento após o train_test_split possui uma ordem aleatória diferente do conjunto de dados original. A figura “Aplicação train_test_split” exibe o resultado desse processo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
Tamanho do conjunto de dados original: (178, 13)
Tamanho do conjunto de treinamento: (124, 13)
Tamanho do conjunto de teste: (54, 13)
Conjunto de dados antes do split:
```

	target
0	0
1	0
2	0
3	0
4	0

dtype: int64

```
Conjunto de treinamento:
```

	target
138	2
104	1
78	1
36	0
93	1

dtype: int64

Figura 15 - Aplicação train_test_split
Fonte: Elaborado pelo autor (2024)

Dica:

- Validação cruzada (cross_val_score e KFold), que ajudam a avaliar a robustez do modelo.

- Busca de hiperparâmetros (GridSearchCV e RandomizedSearchCV) permitem otimizar hiperparâmetros de modelos para melhor desempenho.

Pronto, agora os nossos dados estão prontos para serem utilizados no treinamento de modelos de aprendizado de máquina. Na próxima seção, será abordado os principais estimadores que o scikit-learn disponibiliza em sua biblioteca, e como é simples de usar em nosso projeto de dados.

1.8 Estimadores e treinamento de modelos

No scikit-learn, o termo "*estimators*" (ou estimadores) são modelos ou algoritmos que podem aprender a partir de dados (treinamento) e fazer previsões (predição), como ilustra a figura "Utilização dos métodos fit predict". Em outras palavras, um estimador é qualquer objeto que implemente os métodos fit() e predict(), que são:

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

- **fit(X, y):** treina o modelo (estimador) a partir dos dados de entrada X e dos rótulos y.
- **predict(X):** realiza previsões com base nos dados de entrada X após o treinamento.

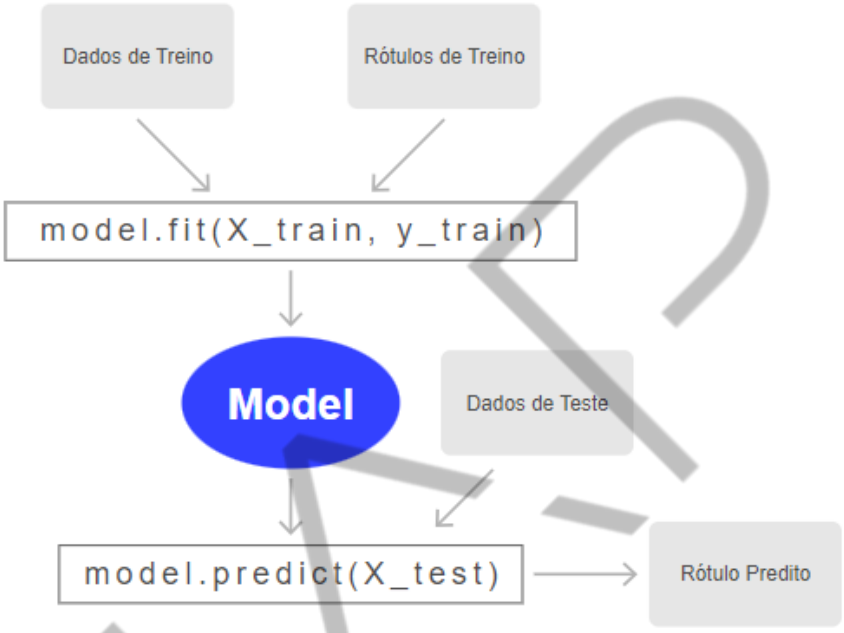

Figura 16 - Utilização dos métodos fit predict
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2024)

Os estimadores são a base de qualquer algoritmo de aprendizado supervisionado ou não supervisionado. E podem ser divididos em várias categorias:

- **Modelos de Regressão:** como LinearRegression e Ridge.
- **Modelos de Classificação:** como LogisticRegression, SVC (Support Vector Classifier) e RandomForestClassifier.
- **Algoritmos de Clusterização:** como KMeans e DBSCAN.
- **Redução de Dimensionalidade:** como PCA (Principal Component Analysis).

O método score() é utilizado no scikit-learn para avaliar o desempenho de um modelo, mas o comportamento desse método varia de acordo com o tipo de modelo

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

que está sendo utilizado (classificação, regressão, clusterização, entre outros). Veja a figura “Utilização dos métodos score”.

- **Problema de classificação:** o método score() retorna a acurácia do modelo, que é a proporção de previsões corretas.
- **Problema de regressão:** retorna o coeficiente de determinação (R^2), que mede o quão bem os valores previstos se ajustam aos valores reais.

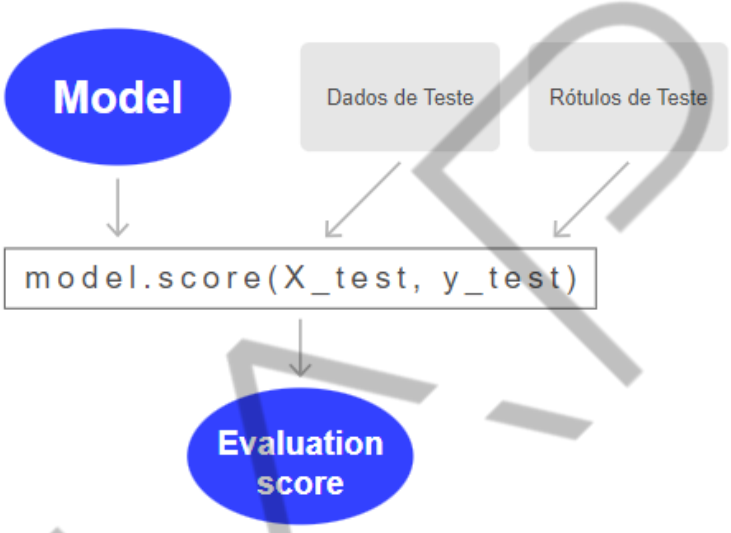

Figura 17 - Utilização dos métodos score
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2024)

O foco deste capítulo está na aplicação das técnicas de Aprendizado de Máquina por meio da biblioteca scikit-learn em diferentes tipos de problemas.

- **Estimadores de classificação:** são utilizados para problemas onde o objetivo é prever rótulos discretos ou categorias. Os mais conhecidos são:
  - **sklearn.linear_model:** Regressão Logística (LogisticRegression) para problemas de classificação binária ou multiclasse.
  - **sklearn.ensemble:** RandomForestClassifier, AdaBoostClassifier, e outros métodos de ensemble.
  - **sklearn.svm:** máquinas de vetores de suporte (SVM) para classificação.
  - **sklearn.neighbors:** classificação com K-Nearest Neighbors (KNN).

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Vamos utilizar o dataset wine para treinar um classificador de vinhos com base em suas características. Primeiro, iremos carregar e preparar os dados para serem utilizados pelo estimador.

```
import pandas as pd
from sklearn.datasets import load_wine
from sklearn.model_selection import train_test_split

# Carregando novamente o dataset
wine = load_wine()
X = wine.data  # Características
y = wine.target  # Rótulos

# Convertendo para DataFrame para melhor visualização
df = pd.DataFrame(X, columns=wine.feature_names)
df['target'] = y

# Escolhendo X e y para fazer o split
X = df.drop('target', axis=1)
y = df['target']

# Dividindo o DataFrame em 70% para treino e 30% para
teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

Código-fonte 15 - Dataset Wine para classificação
Fonte: Elaborado pelo autor (2024)

Com o conjunto de dados pronto, podemos seguir para a etapa de treinamento. Nesta etapa, iremos utilizar um RandomForestClassifier (mas poderia ser qualquer outro algoritmo de classificação, pois a forma de utilização não muda). A função fit(), recebe X_train e y_train como entradas para o treinamento do modelo. Após rodar o fit() o modelo estará treinado. Para realizar uma predição, devemos chamar o método predict() utilizando uma entrada com o mesmo número de atributos das amostras alocadas em X_tain, neste caso, vamos usar X_test que retorna uma predição para cada entrada. O método score(), por sua vez, compara os resultados preditos *versus* os rótulos reais, e calcula a acurácia do modelo treinado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.ensemble import RandomForestClassifier

# Instanciando o modelo
model = RandomForestClassifier()

# Treinamento do modelo
model.fit(X_train, y_train)

# Predição
y_pred = model.predict(X_test)

# Avaliando a acurácia do modelo usando o método score()
accuracy = model.score(X_test, y_test)
print(f"Acurácia do modelo: {accuracy:.2f}")
```

Código-fonte 16 - Treinamento dataset Wine para classificação
Fonte: Elaborado pelo autor (2024)

Uma das grandes vantagens do scikit-learn é a sua interface unificada para todos os estimadores. Isso significa que, uma vez que você tenha organizado seus dados e feito a divisão entre os conjuntos de treino e teste, você pode facilmente trocar entre diferentes modelos (estimadores) com apenas pequenas alterações no código. Os métodos principais, como fit(), predict() e score(), estão presentes em praticamente todos os estimadores no scikit-learn, independentemente de serem algoritmos de classificação, regressão ou clusterização. Isso torna o processo de experimentação e comparação entre diferentes modelos extremamente simples.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.ensemble import RandomForestClassifier
from sklearn.svm import SVC
from sklearn.neighbors import KNeighborsClassifier

# Testando diferentes algoritmos de classificação

# Random Forest
model_rf = RandomForestClassifier()
model_rf.fit(X_train, y_train)
accuracy_rf = model_rf.score(X_test, y_test)
print(f"Acurácia com Random Forest: {accuracy_rf:.2f}")

# SVM
model_svm = SVC()
model_svm.fit(X_train, y_train)
accuracy_svm = model_svm.score(X_test, y_test)
print(f"Acurácia com SVM: {accuracy_svm:.2f}")

# KNN
model_knn = KNeighborsClassifier()
model_knn.fit(X_train, y_train)
accuracy_knn = model_knn.score(X_test, y_test)
print(f"Acurácia com KNN: {accuracy_knn:.2f}")
```

Código-fonte 17 - Testando diferentes algoritmos de classificação
Fonte: Elaborado pelo autor (2024)

- **Estimadores de regressão:** são utilizados para prever valores contínuos. Os mais conhecidos são:
  - **sklearn.linear_model:** Regressão Linear, Lasso, Ridge e outros modelos lineares.
  - **sklearn.ensemble:** métodos de ensemble para regressão, como Random Forest Regressor e Gradient Boosting.
  - **sklearn.svm:** Suporte Vetorial para Regressão (SVR).
  - **sklearn.neighbors:** regressão baseada em vizinhos mais próximos.

Utilizaremos o conjunto de dados California Housing, que contém informações sobre várias características das casas na Califórnia e o nosso *target* é realizar a predição do valor médio das casas. Como o treinamento do modelo é o foco principal, vamos pular a etapa de pré-processamento e de análise de dados. No código a seguir, realizamos o importe do dataset, convertemos para dataFrame do Pandas e fazemos a divisão dos dados em treino e teste.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split

# Carregar o dataset California Housing
housing = fetch_california_housing()
X = pd.DataFrame(housing.data,
columns=housing.feature_names)
y = pd.Series(housing.target, name='MedHouseVal')

# Dividindo os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)
```

Código-fonte 18 - Dataset California para regressão
Fonte: Elaborado pelo autor (2024)

Em seguida, podemos aplicar os estimadores de regressão para criar os modelos para predição de valores de imóveis.

```
from sklearn.linear_model import LinearRegression, Ridge
from sklearn.ensemble import RandomForestRegressor

# Testando diferentes algoritmos de regressão

# Regressão Linear
model_lr = LinearRegression()
model_lr.fit(X_train, y_train)
r2 = model_lr.score(X_test, y_test)
print(f"Performance do modelo: {r2:.2f}")

# Regressão Ridge
model_ridge = Ridge(alpha=1.0)
model_ridge.fit(X_train, y_train)
r2 = model_ridge.score(X_test, y_test)
print(f"Performance do modelo: {r2:.2f}")

# Regressão com Random Forest
model_rf = RandomForestRegressor()
model_rf.fit(X_train, y_train)
r2 = model_rf.score(X_test, y_test)
print(f"Performance do modelo: {r2:.2f}")
```

Código-fonte 19 - Testando diferentes algoritmos de regressão
Fonte: Elaborado pelo autor (2024)

Esperamos como resultado que o RandomForestRegressor tenha um desempenho superior em relação aos demais modelos lineares, como

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

LinearRegression e Ridge, pois é capaz de capturar interações mais complexas entre as variáveis.

- **Estimadores de clusterização (não supervisionado):** são utilizados para agrupar dados sem a necessidade de rótulos (alvos) e são amplamente aplicados em tarefas como análise de grupos em dados, segmentação de dados e outras tarefas exploratórias.
  - **sklearn.cluster:** K-Means, DBSCAN e outros algoritmos de clusterização.
  - **sklearn.mixture:** modelos de mistura gaussiana para aprendizado não supervisionado.

Iremos desenvolver um conjunto de dados sintéticos utilizando a função make_blobs, que gera grupos (blobs) de pontos com as características de 750 pontos distribuídos em 3 centros com dispersão de 0.4.

```
from sklearn.datasets import make_blobs
from sklearn.preprocessing import StandardScaler
import matplotlib.pyplot as plt

centers = [[1, 1], [-1, -1], [1, -1]]
X, labels_true = make_blobs(
    n_samples=750,   centers=centers,   cluster_std=0.4,
random_state=0
)

X_scaled = StandardScaler().fit_transform(X)

# Exibe os dados
plt.scatter(X[:, 0], X[:, 1])
plt.show()
```

Código-fonte 20 - Dataset make_blob
Fonte: Elaborado pelo autor (2024)

Com o Matplotlib podemos visualizar o conjunto de dados criado.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

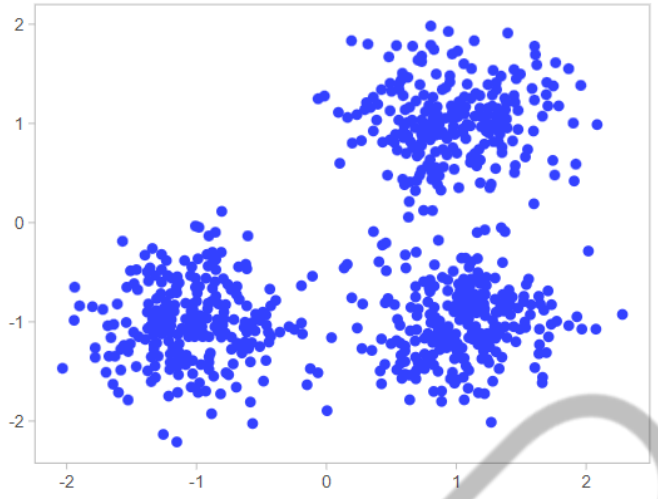

Figura 18 - Dados make_blob
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2024)

Neste momento, podemos aplicar as técnicas de agrupamento K-Means e DBSCAN para encontrar os grupos de dados existentes.

```
from sklearn.cluster import KMeans, DBSCAN
import seaborn as sns

# Aplicando K-Means com 3 clusters
kmeans = KMeans(n_clusters=3, random_state=42)
kmeans_labels = kmeans.fit_predict(X_scaled)

# Aplicando DBSCAN
dbscan = DBSCAN(eps=0.3, min_samples=10)
dbscan_labels = dbscan.fit_predict(X_scaled)

# Visualizando os clusters criados pelo K-Means
plt.figure(figsize=(12, 6))

# Gráfico para K-Means
plt.subplot(1, 2, 1)
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1],
hue=kmeans_labels, palette='viridis', s=50)
plt.title('Clusters criados pelo K-Means')

# Gráfico para DBSCAN
plt.subplot(1, 2, 2)
sns.scatterplot(x=X_scaled[:, 0], y=X_scaled[:, 1],
hue=dbscan_labels, palette='viridis', s=50)
plt.title('Clusters criados pelo DBSCAN')

plt.show()
```

Código-fonte 21 - Testando diferentes algoritmos de agrupamento
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

O algoritmo K-Means foi inicializado com três clusters, pois sabemos que nossos dados sintéticos possuem três grupos. O algoritmo DBSCAN identifica clusters com base na densidade dos dados, sem precisar saber o número de clusters previamente. Ele também detecta outliers, que são pontos que não pertencem a nenhum cluster e são atribuídos com rótulos de -1, como exibe a figura “Resultado Agrupamento K-Means e DBSCAN”.

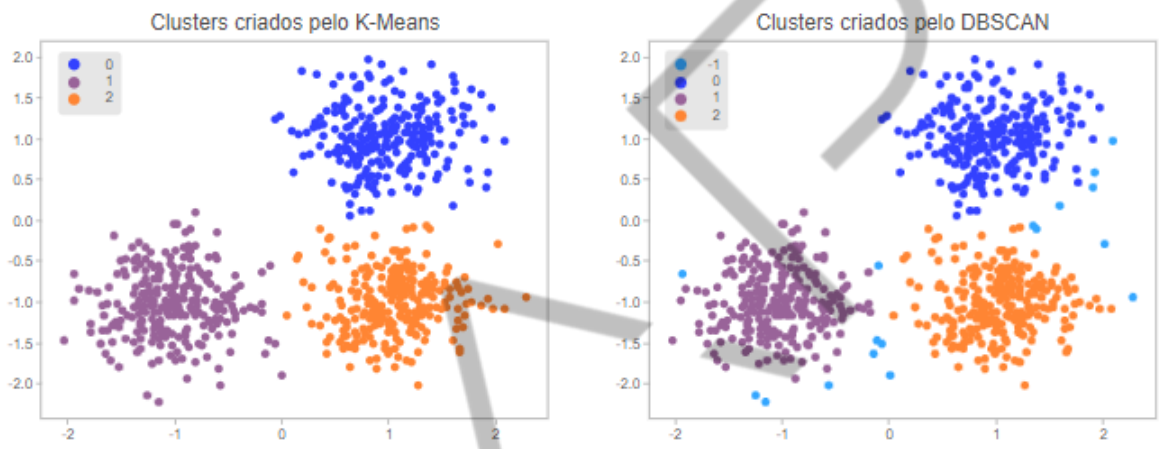

Figura 19 - Resultado Agrupamento K-Means e DBSCAN
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2024)

Saiba mais:

Conheça outros estimadores de aprendizado não supervisionado com os métodos de redução de dimensionalidade:

- PCA (análise de componentes principais);

- TSNE.

1.9 Ferramentas para avaliação e otimização

A avaliação de modelos é uma etapa crucial em qualquer pipeline de Machine Learning. Dependendo do tipo de tarefa (classificação, regressão ou clusterização), diferentes métricas são utilizadas para medir o quão bem o modelo está desempenhando. A seguir, exploraremos as principais métricas oferecidas pelo scikit-learn.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Acurácia ou taxa de acerto é a métrica mais utilizada em problemas de classificação, pois mede o número de acertos do modelo dividido pelo número total testado. Por exemplo, se o modelo possui uma acurácia de 0.95 (95%), o seu modelo acerta 95 de 100 previsões. Essa é a métrica mais simples e mais utilizada em datasets onde as classes são balanceadas, ou seja, possuímos proporções parecidas de amostras para cada uma das categorias.

```
import numpy as np
from sklearn.metrics import accuracy_score # importa
método da acurácia

# Cria dois array de teste
y_pred = [0, 2, 1, 3]
y_true = [0, 1, 2, 3]

# Calcula acurácia
accuracy_score(y_true, y_pred)

# Resultado
# 0.5
```

Código-fonte 22 - Testando diferentes algoritmos de agrupamento (2)
Fonte: Elaborado pelo autor (2024)

Dependendo do problema, pode ser necessário utilizar outras métricas de avaliação do modelo de classificação. A matriz de confusão exibe de forma simples os resultados de classificação onde as linhas representam os valores reais e as colunas os valores preditos, a diagonal principal é composta pelos acertos do modelo e os demais valores são os erros cometidos como pode ser observado na figura “Matriz de confusão”.

		Predito	
		Negativo	Positivo
Real	Negativo	VN	FP
	Positivo	FN	VP

Figura 20 - Matriz de confusão
Fonte: Elaborado pelo autor (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Da matriz de confusão podemos extrair as seguintes análises:

- **Verdadeiro Positivo (VP):** quando o modelo prediz que é positivo e realmente é positivo.
- **Verdadeiro Negativo (VN):** quando o modelo prediz que é negativo realmente é negativo.
- **Falso Positivo (FP):** quando o modelo prediz que é positivo, mas na verdade é negativo.
- **Falso Negativo (FN):** quando o modelo prediz que é negativo, mas na verdade é positivo.

A partir da matriz de confusão extraímos diversas medidas de performance, tais como: F1-Score, Recall, Precision e Especificidade. Iremos utilizar o dataset wines treinado com KNN para visualizar essas métricas de performance.

```
from sklearn.model_selection import train_test_split
from sklearn.datasets import load_wine
import pandas as pd

# Carregando novamente o dataset Wine
wine = load_wine()
X = wine.data  # Características
y = wine.target  # Rótulos

# Convertendo para DataFrame para melhor visualização
df = pd.DataFrame(X, columns=wine.feature_names)
df['target'] = y

# Dividindo o conjunto de dados em 70% treino e 30% teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# KNN
model_knn = KNeighborsClassifier()
model_knn.fit(X_train, y_train)
y_pred = model_knn.predict(X_test)
```

Código-fonte 23 - Treina com knn o wines
Fonte: Elaborado pelo autor (2024)

Após treinar o modelo, vamos utilizar as métricas de performance. Observe que cada métrica tem um papel específico, e a matriz de confusão oferece uma visão detalhada do desempenho do modelo.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
# Importando todas as métricas necessárias
from sklearn.metrics import accuracy_score, precision_score, recall_score, f1_score, confusion_matrix

# Calculando e exibindo a Acurácia
accuracy = accuracy_score(y_test, y_pred)
print(f"Acurácia: {accuracy:.2f}")

# Calculando e exibindo a Precisão
precision = precision_score(y_test, y_pred, average='weighted')
print(f"Precisão (Weighted): {precision:.2f}")

# Calculando e exibindo o Recall
recall = recall_score(y_test, y_pred, average='weighted')
print(f"Recall (Weighted): {recall:.2f}")

# Calculando e exibindo o F1-Score
f1 = f1_score(y_test, y_pred, average='weighted')
print(f"F1-Score (Weighted): {f1:.2f}")

# Calculando e exibindo a Matriz de Confusão
matrix = confusion_matrix(y_test, y_pred)
print("Matriz de Confusão:")
print(matrix)
```

Código-fonte 24 - Métricas de performance
Fonte: Elaborado pelo autor (2024)

Em problemas de regressão predizemos valores numéricos, nesse sentido, calculamos o erro que é basicamente a diferença entre o valor real (y) e o valor predito(y), como mostra a figura “Matriz de confusão”.

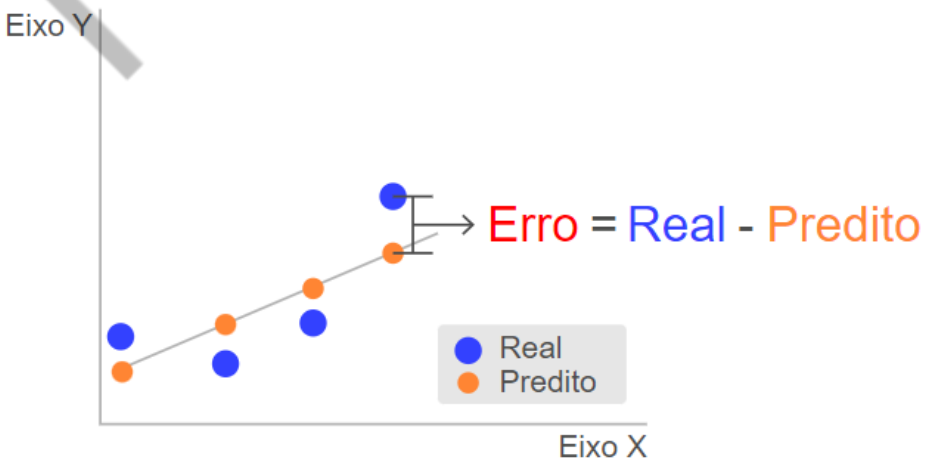

Figura 21 - Matriz de confusão (2)
Fonte: Elaborado pelo autor (2024), adaptado por FIAP (2024)

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

Nesta próxima etapa, aplicamos as métricas de performance para o nosso exemplo de predição de imóveis na Califórnia.

```
import pandas as pd
from sklearn.datasets import fetch_california_housing
from sklearn.model_selection import train_test_split
from sklearn.ensemble import RandomForestRegressor

# Carregar o dataset California Housing
housing = fetch_california_housing()
X = pd.DataFrame(housing.data,
columns=housing.feature_names)
y = pd.Series(housing.target, name='MedHouseVal')

# Dividindo os dados em treino e teste
X_train, X_test, y_train, y_test = train_test_split(X, y,
test_size=0.3, random_state=42)

# Regressão com Random Forest
model_rf = RandomForestRegressor()
model_rf.fit(X_train, y_train)
y_pred = model_rf.predict(X_test)
```

Código-fonte 25 - Treina com Random Forest Regressor o California
Fonte: Elaborado pelo autor (2024)

Ao utilizar o RandomForestRegressor para treinar o modelo, podemos aplicar as métricas de performance para problemas de regressão, em que:

- **mean_squared_error (MSE):** mede a média dos quadrados dos erros, penalizando erros grandes.
- **RMSE:** a raiz quadrada do MSE, que traz os erros para a mesma unidade das variáveis.
- **mean_absolute_error (MAE):** a média das diferenças absolutas entre predições e valores reais.
- **mean_absolute_percentage_error (MAPE):** representa o erro em termos percentuais.
- **R^2 (Coeficiente de Determinação):** mede a qualidade do ajuste do modelo, sendo 1 o ajuste perfeito.

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

```
from sklearn.metrics import mean_squared_error, mean_absolute_error, mean_absolute_percentage_error, r2_score

# Calculando as métricas
mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
mae = mean_absolute_error(y_test, y_pred)
mape = mean_absolute_percentage_error(y_test, y_pred)
r2 = r2_score(y_test, y_pred)

# Exibindo as métricas
print(f"Erro Quadrático Médio (MSE): {mse:.2f}")
print(f"Raiz do Erro Quadrático Médio (RMSE): {rmse:.2f}")
print(f"Média do Erro Absoluto (MAE): {mae:.2f}")
print(f"Erro Percentual Médio Absoluto (MAPE): {mape:.2f}")
print(f"Coeficiente de Determinação (R²): {r2:.2f}")
```

Código-fonte 26 - Treina com Random Forest Regressor o California (2)
Fonte: Elaborado pelo autor (2024)

FIAP

CONCLUSÃO

Neste capítulo, estabelecemos uma base sólida e consistente da biblioteca scikit-learn em projetos de Machine Learning. Desde a manipulação de conjuntos de dados, passando por pré-processamento, treinamento de modelos até a avaliação de métricas. Além disso, aprendemos que o scikit-learn oferece ferramentas de fácil utilização para solucionar problemas de aprendizado supervisionado e não supervisionado.

A flexibilidade dos estimadores no scikit-learn permite testar diferentes algoritmos com mudanças mínimas no código. Por essa razão, é extremamente importante avaliar adequadamente se o desempenho dos modelos foi enfatizado, utilizando métricas apropriadas para classificação, regressão e clusterização.

O scikit-learn oferece muito mais do que exploramos até o momento. Além das ferramentas mencionadas, ele possui funcionalidades avançadas como métodos de redução de dimensionalidade, técnicas mais sofisticadas de otimização de hiperparâmetros e modelos mais complexos, como redes neurais. É interessante que você continue explorando a documentação oficial e experimente novas técnicas para se aprofundar ainda mais no Machine Learning com scikit-learn.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

REFERÊNCIAS

GONZÁLEZ, Matheus. CRISP-DM na prática. **Medium**, 23 de junho de 2019. Disponível em: <https://medium.com/@mgonzaleze/crisp-dm-na-pr%C3%A1tica-65be0ee92ada>. Acesso em: 19 set. 2024.

SCIKIT-LEARN. **scikit-learn.org**, 2024. Disponível em: <https://scikit-learn.org/stable/>. Acesso em: 19 set. 2024.

WASKOM, M. L., Seaborn: Statistical Data Visualization. **Journal of Open Source Software**, 60, 2021, p. 3021-3025.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com

GLOSSÁRIO

Termo	Explicação
CRISP	Cross Industry Standard Process for Data Mining.
ML	Machine Learning.

FIAP

PDF exclusivo para Francismar Alves Martins Júnior - rm562869
yggdrasil.git@gmail.com